SY B.Tech. – ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING
PRACTICAL ASSIGNMENT 09

1. Student Details

Student Name	Homesh Sawarkar
PRN	202501110069
Division	A
Batch	A2
Roll No.	202501110069
Date of Submission	27/10/2026

2. Practical Details

Practical No.	09
Practical Title	Std::Move In Utility
Date Performed	-
Faculty Name	Khushal Khairnar

3. Problem Statement

Write a program on std::move in utility.

4. Objectives

1. To understand the concept of std::move and its use in the C++ <utility> library.
2. To learn how std::move enables move semantics and helps transfer resources between objects.
3. To demonstrate the difference between copying and moving objects in C++.

5. Expected Learning Outcomes

After completing this practical, the student will be able to:

1. Explain the purpose and working of std::move in C++.
2. Implement move constructors and use std::move to transfer resources between objects.
3. Differentiate between copy operations and move operations and understand their benefits in memory and resource management.

6. Theory / Concept

std::move is a utility function provided by the C++ Standard Library in the <utility> header file. It allows an object to be treated as an rvalue, enabling move operations when the type supports them.

Move Semantics: Move semantics allow resources, such as dynamically allocated memory, to be transferred from one object to another instead of copying them. This can improve performance and reduce unnecessary resource duplication.

Move Constructor: A move constructor initializes a new object by transferring resources from an existing object, usually an rvalue reference of the form ClassName&&.

Copy vs. Move: Copying generally duplicates data, while moving can transfer ownership of existing resources. std::move itself does not move data; it enables the compiler to select a suitable move constructor or move assignment operator when available.

7. Algorithm / Pseudocode

START

Step 1: Include the <iostream>, <string>, and <utility> header files.

Step 2: Define a class containing a string data member and implement a constructor to initialize it.

Step 3: Define a move constructor that transfers the string resource from the source object using std::move.

Step 4: Create an object and use std::move to initialize another object from it, invoking the move constructor.

Step 5: Display the contents of both objects before and after the move to observe the transfer of resources.

STOP

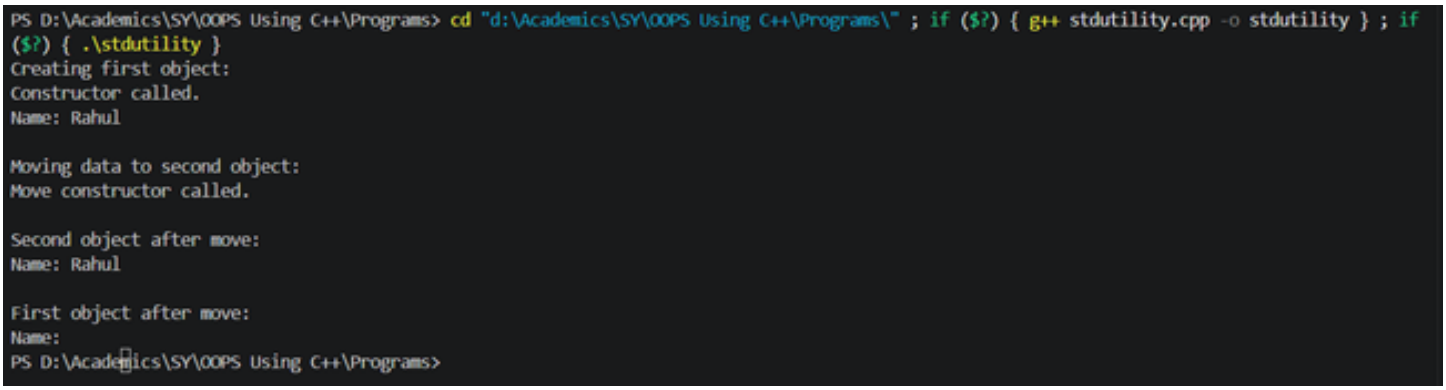
08. Technology / Tools / Libraries Used

Technology / Tool	**Purpose**
Programming Language	C++
IDE / Platform	VS Code
Libraries / Packages	-

09. Implementation / GitHub Repository

Particular	Details
GitHub Repository URL	https://github.com/Homeshsawarkar/Assignment-9-C-
Repository Name	Assignment 9 C++
Branch Used	main
Commit ID / Latest Commit	
Date of Last Commit	Oct 27, 2026

10. Results / Observations



11. Conclusion

In this practical, std::move from the C++ <utility> library was studied and implemented using a move constructor. The program demonstrated how resources can be transferred from one object to another instead of being copied. This practical helped develop an understanding of move semantics, resource management, and performance optimization in C++.**12. Student Declaration**

I hereby declare that the above practical work has been completed by me and that the implementation, analysis, and answers are my own work.

Student Name	PRN	Signature
Homesh Sawarkar	202501110069	Homesh A.S